



**Министерство науки и высшего образования Российской Федерации  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
«Московский государственный технический университет  
имени Н.Э. Баумана  
(национальный исследовательский университет)»  
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ Информатика, искусственный интеллект и системы управления

КАФЕДРА Системы обработки информации и управления

# ОТЧЕТ

по Лабораторной работе № 4.3

по дисциплине «**МЕТОДЫ МАШИННОГО ОБУЧЕНИЯ**»

Тема: «Решение задач многоэкстремальной оптимизации»

Студент

ИУ5И-22М

(группа)

(подпись, дата)

Тавуз Мохамад

(И.О. Фамилия)

Преподаватель

Ю.Е.Гапанюк

(подпись, дата)

(И.О. Фамилия)

2026 г.

## 1) Введение

Методы многоэкстремальной оптимизации играют важную роль в задачах математического программирования и машинного обучения, так как многие практические целевые функции имеют несколько локальных экстремумов. В таких задачах обычные локальные методы оптимизации могут сходиться к ближайшему локальному минимуму и не гарантируют нахождение глобального решения.

Цель данной лабораторной работы — изучение и программная реализация методов решения задач многоэкстремальной оптимизации. В ходе работы рассматриваются два подхода: метод случайного мултистарта для поиска глобального минимума функции Шекеля и генетический алгоритм для решения задачи оптимизации заданной многоэкстремальной функции.

В первой части работы реализуется метод мултистарта, основанный на запуске локальной оптимизации из нескольких случайных начальных точек. Дополнительно выполняется кластеризация найденных точек методом ближайшего соседа. Во второй части реализуется генетический алгоритм с операциями отбора, кроссинговера и мутации, а также его модификация с сохранением лучшей особи. Для всех найденных решений выполняется проверка допустимости и формируется сравнительная таблица результатов.

## 2) Описание задания

В соответствии с заданием лабораторной работы необходимо решить задачу многоэкстремальной оптимизации двумя способами: методом случайного мултистарта и генетическим алгоритмом.

В первой части работы требуется найти глобальный минимум функции Шекеля. Для варианта 2 заданы следующие параметры:  $\alpha = 2$ ,  $\beta = 8$ ,  $a = 1$ ,  $b = 3$ ,  $c = 2$ . Область допустимых значений имеет вид  $x_i \in [2, 8]$ ,  $i = 1, 2, 3$ . Для решения задачи необходимо реализовать метод мултистарта, выполнить локальную оптимизацию из нескольких

случайных начальных точек, выбрать лучшее найденное решение и проверить его допустимость.

Во второй части работы необходимо реализовать генетический алгоритм для заданной многоэкстремальной функции. Для варианта 2 используется двумерная функция, определенная на области  $x_i \in [0, 5]$ ,  $i = 1, 2$ . В ходе работы необходимо сформировать начальную популяцию, определить функцию приспособленности, реализовать операции отбора, кроссинговера и мутации, а затем найти приближенное решение задачи минимизации. Дополнительно рассматривается вариант генетического алгоритма с сохранением лучшей особи.

По результатам вычислений необходимо представить найденные точки, значения целевых функций, время выполнения, количество итераций или поколений, а также выполнить проверку допустимости найденных решений.

### **3) Текст программы и экранные формы с примерами выполнения**

В данном разделе приведены основные этапы программной реализации методов многоэкстремальной оптимизации. Все вычисления были выполнены в среде Google Colab с использованием языка Python.

Для численных вычислений использовалась библиотека `numpy`, для формирования таблиц результатов — `pandas`, для построения графиков — `matplotlib`. Также была использована функция `minimize` из модуля `scipy.optimize`, применяемая для выполнения локальной оптимизации в методе случайного мултистарта. Для работы с расстояниями между найденными точками была подключена функция `cdist` из `scipy.spatial.distance`.

```
▶ # Импорт необходимых библиотек для выполнения лабораторной работы

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time

from scipy.optimize import minimize
from scipy.spatial.distance import cdist

np.random.seed(42)

print("Libraries imported successfully")
```

---

```
... Libraries imported successfully
```

Рисунок 1 — Импорт необходимых библиотек

### 3.1. Задание параметров варианта и определение функции Шекеля

На первом этапе была рассмотрена задача поиска глобального минимума функции Шекеля методом случайного мултистарта. Для выполнения вычислений был выбран вариант 2. В соответствии с вариантом были заданы параметры  $\alpha = 2$ ,  $\beta = 8$ ,  $a = 1$ ,  $b = 3$ ,  $c = 2$ , а также размерность задачи  $n = 3$ .

Область допустимых значений переменных задается отрезком  $[2, 8]$  для каждой координаты. После задания параметров была реализована функция Шекеля. Для предварительной проверки корректности реализации было вычислено значение функции в тестовой точке  $x = (3, 3, 3)$ . Полученное значение  $f(x) = -1.5$  далее используется как ориентир при анализе результатов оптимизации.

▶ # Задание параметров варианта и определение функции Шекеля

```
variant = 2

alpha = 2
beta = 8

a = np.array([1.0, 1.0, 1.0])
b = np.array([3.0, 3.0, 3.0])
c = np.array([2.0, 2.0, 2.0])

lower_bound = alpha
upper_bound = beta
n = 3

def shekel_function(x):
    x = np.array(x, dtype=float)
    value = 0.0

    for i in range(n):
        inner_sum = 0.0
        for j in range(n):
            inner_sum += a[j] * (x[j] - b[j]) ** 2
        value += 1.0 / (inner_sum + c[i])

    return -value

x_test = np.array([3.0, 3.0, 3.0])
f_test = shekel_function(x_test)

print("Variant:", variant)
print("alpha =", alpha)
print("beta =", beta)
print("a =", a)
print("b =", b)
print("c =", c)
print("Dimension n =", n)
print("Test point:", x_test)
print("Function value at test point:", f_test)
```

---

```
... Variant: 2
alpha = 2
beta = 8
a = [1. 1. 1.]
b = [3. 3. 3.]
c = [2. 2. 2.]
Dimension n = 3
Test point: [3. 3. 3.]
Function value at test point: -1.5
```

Рисунок 2 — Задание параметров варианта и функции Шекеля

### 3.2. Проверка допустимости тестовых точек

После определения функции Шекеля была выполнена проверка допустимости точек в заданной области поиска. Для этого были реализованы функции `is_feasible` и `boundary_violation`. Первая функция проверяет принадлежность точки области  $[2, 8]$  по каждой координате, а вторая вычисляет величину нарушения границ области допустимых решений.

Для проверки были рассмотрены несколько тестовых точек. Точки  $(3, 3, 3)$ ,  $(2, 2, 2)$  и  $(8, 8, 8)$  принадлежат допустимой области, поэтому для них значение нарушения границ равно нулю. Точки  $(1, 3, 3)$  и  $(9, 3, 3)$  находятся за пределами допустимой области, поэтому для них функция проверки допустимости возвращает значение `False`.

```
 # Проверка допустимости точек в заданной области поиска

def is_feasible(x):
    x = np.array(x, dtype=float)
    return np.all(x >= lower_bound) and np.all(x <= upper_bound)

def boundary_violation(x):
    x = np.array(x, dtype=float)
    lower_violation = np.maximum(lower_bound - x, 0)
    upper_violation = np.maximum(x - upper_bound, 0)
    return np.sum(lower_violation ** 2 + upper_violation ** 2)

test_points = [
    np.array([3.0, 3.0, 3.0]),
    np.array([2.0, 2.0, 2.0]),
    np.array([8.0, 8.0, 8.0]),
    np.array([1.0, 3.0, 3.0]),
    np.array([9.0, 3.0, 3.0])
]

for point in test_points:
    print("Point:", point)
    print("f(x):", shekel_function(point))
    print("Boundary violation:", boundary_violation(point))
    print("Feasible:", is_feasible(point))
    print("-" * 50)
```

Рисунок 3 — Код проверки допустимости тестовых точек

```

... Point: [3. 3. 3.]
    f(x): -1.5
    Boundary violation: 0.0
    Feasible: True
-----
    Point: [2. 2. 2.]
    f(x): -0.6000000000000001
    Boundary violation: 0.0
    Feasible: True
-----
    Point: [8. 8. 8.]
    f(x): -0.03896103896103896
    Boundary violation: 0.0
    Feasible: True
-----
    Point: [1. 3. 3.]
    f(x): -0.5
    Boundary violation: 1.0
    Feasible: False
-----
    Point: [9. 3. 3.]
    f(x): -0.07894736842105263
    Boundary violation: 1.0
    Feasible: False
-----

```

Рисунок 4 — Результат проверки допустимости тестовых точек

### 3.3. Выполнение локальной оптимизации функции Шекеля

Перед запуском метода случайного мултистарта была выполнена локальная оптимизация функции Шекеля из одной начальной точки. В качестве начального приближения была выбрана точка  $x_0 = (5, 5, 5)$ , принадлежащая допустимой области  $[2, 8]$ .

Для локального поиска использовался метод L-BFGS-B, так как он поддерживает граничные ограничения на переменные. В результате локальной оптимизации была найдена точка  $x = (3, 3, 3)$ , в которой значение целевой функции равно  $f(x) = -1.5$ . Найденная точка удовлетворяет ограничениям области поиска, а величина нарушения границ равна нулю.

```

▶ # Выполнение локальной оптимизации функции Шекеля из одной начальной точки

bounds = [(lower_bound, upper_bound) for _ in range(n)]

x0_local = np.array([5.0, 5.0, 5.0])

start_time = time.time()

local_result = minimize(
    shekel_function,
    x0_local,
    method="L-BFGS-B",
    bounds=bounds,
    options={
        "maxiter": 1000,
        "ftol": 1e-12,
        "gtol": 1e-8
    }
)

local_time = time.time() - start_time

print("Initial point:", x0_local)
print("Optimization success:", local_result.success)
print("Message:", local_result.message)
print("Found point:", local_result.x)
print("Function value:", local_result.fun)
print("Number of iterations:", local_result.nit)
print("Execution time:", local_time)
print("Feasible:", is_feasible(local_result.x))
print("Boundary violation:", boundary_violation(local_result.x))

```

Рисунок 5 — Код локальной оптимизации функции Шекеля

```

... Initial point: [5. 5. 5.]
Optimization success: True
Message: CONVERGENCE: RELATIVE REDUCTION OF F <= FACTR*EPSMCH
Found point: [3. 3. 3.]
Function value: -1.5
Number of iterations: 5
Execution time: 0.03519320487976074
Feasible: True
Boundary violation: 0.0

```

Рисунок 6 — Результат локальной оптимизации функции Шекеля



### 3.4. Генерация случайных начальных точек

На следующем этапе были сгенерированы случайные начальные точки для метода мултистарта. Было выбрано 30 начальных точек, равномерно распределенных в допустимой области [2, 8] по каждой координате.

После генерации была выполнена проверка допустимости всех начальных точек. Результат проверки показал, что все сгенерированные точки принадлежат заданной области поиска. Эти точки далее используются как начальные приближения для многократного запуска локальной оптимизации.

```
▶ # Генерация случайных начальных точек для метода мултистарта

num_start_points = 30

initial_points = np.random.uniform(
    low=lower_bound,
    high=upper_bound,
    size=(num_start_points, n)
)

print("Number of generated start points:", num_start_points)
print("Search domain: [{}, {}]" .format(lower_bound, upper_bound))
print("First five generated points:")
print(initial_points[:5])

print("\nFeasibility check for all generated points:")
print(np.all([is_feasible(point) for point in initial_points]))
```

Рисунок 7 — Код генерации случайных начальных точек

```
... Number of generated start points: 30
Search domain: [2, 8]
First five generated points:
[[4.24724071 7.70428584 6.39196365]
 [5.59195091 2.93611184 2.93596712]
 [2.34850167 7.19705687 5.60669007]
 [6.24843547 2.12350697 7.81945911]
 [6.99465584 3.27403466 3.0909498 ]]

Feasibility check for all generated points:
True
```

Рисунок 8 — Результат генерации случайных начальных точек

### 3.5. Реализация метода случайного мультистарта

После генерации начальных точек был реализован метод случайного мультистарта. Основная идея данного метода заключается в многократном запуске локальной оптимизации из различных случайных начальных точек. Такой подход позволяет уменьшить вероятность попадания только в один локальный минимум и повысить надежность поиска глобального минимума.

Для каждой из 30 начальных точек выполнялась локальная оптимизация функции Шекеля методом L-BFGS-B с учетом граничных ограничений. После завершения всех запусков результаты были сохранены в таблицу, содержащую начальные точки, найденные точки минимума, значения целевой функции, количество итераций, признак успешности оптимизации и проверку допустимости найденного решения.

В результате работы метода случайного мультистарта наилучшее найденное решение соответствует точке  $x = (3, 3, 3)$ , а значение функции равно  $f(x) = -1.5$ . Все найденные решения являются допустимыми.

► # Реализация метода случайного мултистарта для функции Шекеля

```
multistart_results = []

start_time = time.time()

for i, x0 in enumerate(initial_points, start=1):
    result = minimize(
        shekel_function,
        x0,
        method="L-BFGS-B",
        bounds=bounds,
        options={
            "maxiter": 1000,
            "ftol": 1e-12,
            "gtol": 1e-8
        }
    )

    multistart_results.append({
        "start_number": i,
        "initial_x1": x0[0],
        "initial_x2": x0[1],
        "initial_x3": x0[2],
        "found_x1": result.x[0],
        "found_x2": result.x[1],
        "found_x3": result.x[2],
        "f_value": result.fun,
        "iterations": result.nit,
        "success": result.success,
        "feasible": is_feasible(result.x)
    })

multistart_time = time.time() - start_time

multistart_df = pd.DataFrame(multistart_results)
best_multistart_result = multistart_df.loc[multistart_df["f_value"].idxmin()]

print("Random multistart completed")
print("Total execution time:", multistart_time)
print("\nBest result:")
print(best_multistart_result)

print("\nFirst ten rows of results table:")
display(multistart_df.head(10))
```

Рисунок 9 — Код реализации метода случайного мултистарта

```

... Random multistart completed
Total execution time: 0.5879275798797607

Best result:
start_number      3
initial_x1      2.348502
initial_x2      7.197057
initial_x3      5.60669
found_x1         3.0
found_x2         3.0
found_x3         3.0
f_value         -1.5
iterations        7
success          True
feasible         True
Name: 2, dtype: object

```

Рисунок 10 — Лучший результат метода случайного мультистарта

First ten rows of results table:

|   | start_number | initial_x1 | initial_x2 | initial_x3 | found_x1 | found_x2 | found_x3 | f_value | iterations | success | feasible |
|---|--------------|------------|------------|------------|----------|----------|----------|---------|------------|---------|----------|
| 0 | 1            | 4.247241   | 7.704286   | 6.391964   | 3.0      | 3.0      | 3.0      | -1.5    | 6          | True    | True     |
| 1 | 2            | 5.591951   | 2.936112   | 2.935967   | 3.0      | 3.0      | 3.0      | -1.5    | 7          | True    | True     |
| 2 | 3            | 2.348502   | 7.197057   | 5.606690   | 3.0      | 3.0      | 3.0      | -1.5    | 7          | True    | True     |
| 3 | 4            | 6.248435   | 2.123507   | 7.819459   | 3.0      | 3.0      | 3.0      | -1.5    | 8          | True    | True     |
| 4 | 5            | 6.994656   | 3.274035   | 3.090950   | 3.0      | 3.0      | 3.0      | -1.5    | 5          | True    | True     |
| 5 | 6            | 3.100427   | 3.825453   | 5.148539   | 3.0      | 3.0      | 3.0      | -1.5    | 6          | True    | True     |
| 6 | 7            | 4.591670   | 3.747375   | 5.671117   | 3.0      | 3.0      | 3.0      | -1.5    | 5          | True    | True     |
| 7 | 8            | 2.836963   | 3.752868   | 4.198171   | 3.0      | 3.0      | 3.0      | -1.5    | 6          | True    | True     |
| 8 | 9            | 4.736420   | 6.711056   | 3.198043   | 3.0      | 3.0      | 3.0      | -1.5    | 5          | True    | True     |
| 9 | 10           | 5.085407   | 5.554487   | 2.278702   | 3.0      | 3.0      | 3.0      | -1.5    | 5          | True    | True     |

Рисунок 11 — Таблица результатов метода случайного мультистарта

### 3.6. Формирование итоговой таблицы метода случайного мультистарта

Для более удобного анализа результатов метода случайного мультистарта была сформирована итоговая таблица. В нее были включены основные характеристики вычислений: количество начальных точек, координаты лучшего найденного решения,

минимальное значение целевой функции, среднее количество итераций, количество успешных запусков, количество допустимых решений и время выполнения алгоритма.

Полученные результаты показывают, что метод случайного мултистарта корректно решил задачу для функции Шекеля. Лучшее найденное решение имеет координаты  $x = (3, 3, 3)$ , значение функции равно  $f(x) = -1.5$ . Все 30 запусков завершились успешно, а все найденные точки удовлетворяют ограничениям допустимой области.

▶ # Формирование итоговой таблицы для метода случайного мултистарта

```
multistart_summary = pd.DataFrame({
    "Method": ["Random Multistart"],
    "Number of start points": [num_start_points],
    "Best x1": [best_multistart_result["found_x1"]],
    "Best x2": [best_multistart_result["found_x2"]],
    "Best x3": [best_multistart_result["found_x3"]],
    "Best f(x)": [best_multistart_result["f_value"]],
    "Average iterations": [multistart_df["iterations"].mean()],
    "Successful runs": [multistart_df["success"].sum()],
    "Feasible solutions": [multistart_df["feasible"].sum()],
    "Execution time": [multistart_time]
})

display(multistart_summary)
```

| *** | Method            | Number of start points | Best x1 | Best x2 | Best x3 | Best f(x) | Average iterations | Successful runs | Feasible solutions | Execution time |
|-----|-------------------|------------------------|---------|---------|---------|-----------|--------------------|-----------------|--------------------|----------------|
| 0   | Random Multistart | 30                     | 3.0     | 3.0     | 3.0     | -1.5      | 5.933333           | 30              | 30                 | 0.587928       |

Рисунок 12 — Итоговая таблица метода случайного мултистарта

### 3.7. Анализ результатов метода случайного мултистарта

Для наглядного анализа результатов метода случайного мултистарта был построен график значений целевой функции, полученных при запуске локальной оптимизации из разных начальных точек. По оси абсцисс отложен номер начальной точки, а по оси ординат — найденное значение функции  $f(x)$ .

Из графика видно, что значения функции для всех запусков практически совпадают и находятся на уровне  $f(x) = -1.5$ . Это означает, что независимо от выбранной случайной начальной точки локальная оптимизация сходится к одной и той же точке минимума функции Шекеля. Таким образом, для выбранного варианта метод случайного мултистарта показал устойчивый результат.

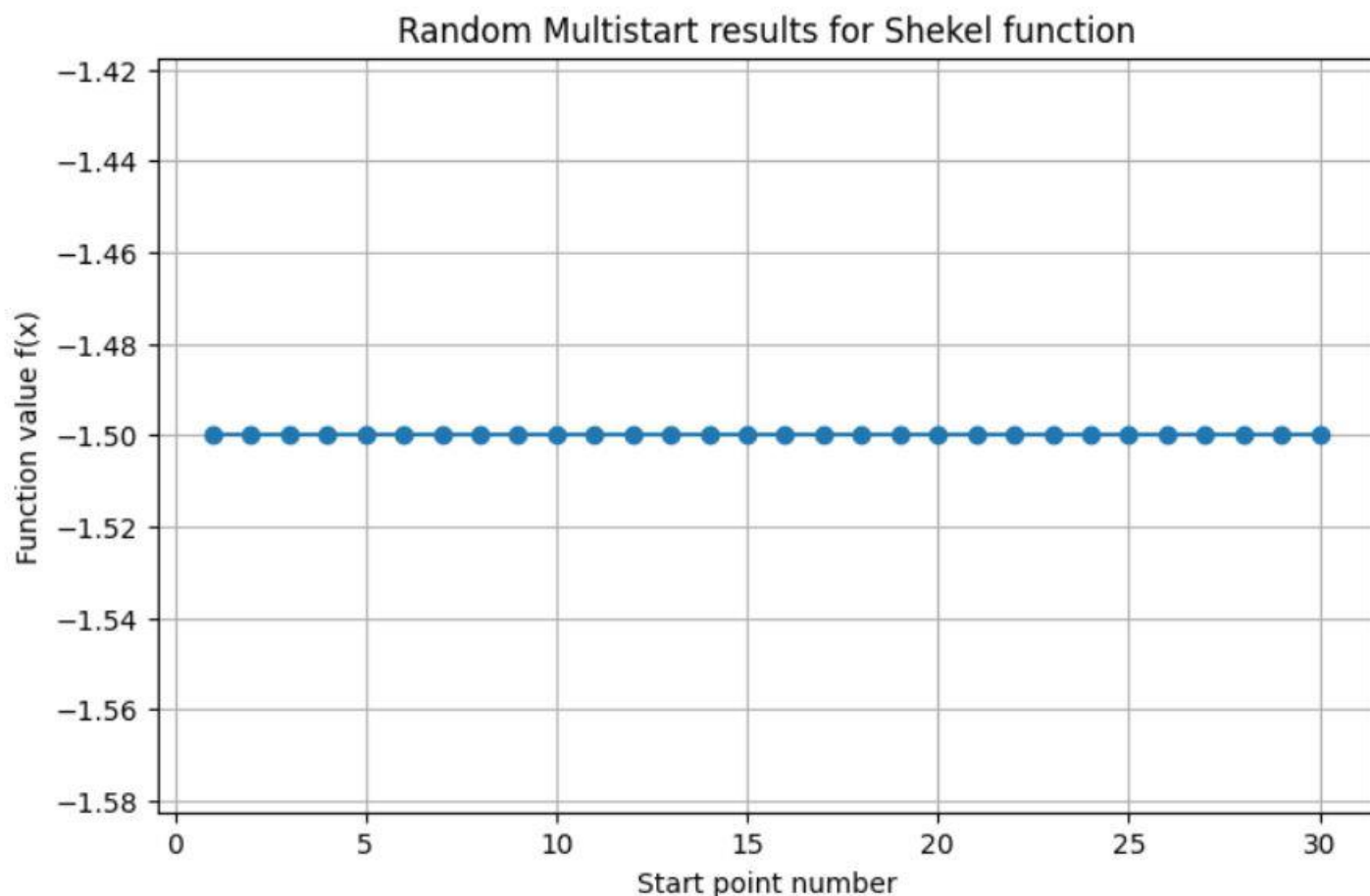


Рисунок 13 — График результатов метода случайного мултистарта

### 3.8. Кластеризация найденных точек методом ближайшего соседа

Дополнительно была выполнена кластеризация найденных точек методом ближайшего соседа. Данный этап используется для анализа того, к скольким различным областям локальных минимумов сходится алгоритм при запуске из разных начальных точек.

В качестве радиуса объединения точек было выбрано значение  $\delta = 10^{-4}$ . Если расстояние между найденными точками не превышает заданный радиус, они относятся к одному кластеру. В результате кластеризации все 30 найденных точек были объединены в один кластер с центром, близким к точке (3, 3, 3).

Полученный результат подтверждает, что все запуски метода мултистарта сошлись к одной и той же области глобального минимума функции Шекеля.



```

# Кластеризация найденных точек методом ближайшего соседа

delta = 1e-4

found_points = multistart_df[["found_x1", "found_x2", "found_x3"]].values

clusters = []

for point in found_points:
    assigned = False

    for cluster in clusters:
        distances = np.linalg.norm(np.array(cluster) - point, axis=1)

        if np.min(distances) <= delta:
            cluster.append(point)
            assigned = True
            break

    if not assigned:
        clusters.append([point])

cluster_summary = []

for i, cluster in enumerate(clusters, start=1):
    cluster_array = np.array(cluster)
    center = cluster_array.mean(axis=0)

    cluster_summary.append({
        "Cluster": i,
        "Number of points": len(cluster),
        "Center x1": center[0],
        "Center x2": center[1],
        "Center x3": center[2]
    })

cluster_df = pd.DataFrame(cluster_summary)

print("Clustering radius delta:", delta)
print("Number of clusters:", len(clusters))
display(cluster_df)

```

```

... Clustering radius delta: 0.0001
Number of clusters: 1

```

|   | Cluster | Number of points | Center x1 | Center x2 | Center x3 |
|---|---------|------------------|-----------|-----------|-----------|
| 0 | 1       | 30               | 3.0       | 3.0       | 3.0       |



Рисунок 14 — Кластеризация найденных точек методом ближайшего соседа

### 3.9. Задание параметров варианта для генетического алгоритма

После выполнения метода случайного мультистарта была рассмотрена вторая часть лабораторной работы — реализация генетического алгоритма. Для вычислений был выбран вариант 2, в котором целевая функция зависит от двух переменных, а область допустимых значений задается отрезком  $[0, 5]$  для каждой координаты.

На данном этапе были заданы параметры варианта, размерность задачи  $n = 2$ , границы области поиска  $\alpha = 0$  и  $\beta = 5$ , а также массивы коэффициентов, необходимые для вычисления целевой функции. Коэффициенты функции были заданы в виде массивов  $a_i$  и  $a_{ij}$  непосредственно в программе в соответствии с выбранным вариантом. После этого была реализована функция `ga_objective_function`, возвращающая значение целевой функции в заданной точке.

Для предварительной проверки корректности реализации было вычислено значение функции в тестовой точке  $x = (2.5, 2.5)$ . Полученное значение используется далее при проверке работы генетического алгоритма.



▶ # Задание параметров варианта для генетического алгоритма и определение целевой функции

```
ga_variant = 2

ga_n = 2
ga_alpha = 0
ga_beta = 5

ga_lower_bound = ga_alpha
ga_upper_bound = ga_beta

a_i = np.arange(1, 11, dtype=float)

a_ij = np.array([
    [1.0, 1.0],
    [2.0, 2.0],
    [3.0, 3.0],
    [4.0, 4.0],
    [5.0, 5.0],
    [1.0, 5.0],
    [5.0, 1.0],
    [2.0, 4.0],
    [4.0, 2.0],
    [3.0, 1.0]
])

def ga_objective_function(x):
    x = np.array(x, dtype=float)
    value = 0.0

    for i in range(10):
        distance_sum = np.sum((x - a_ij[i]) ** 2)
        value += 1.0 / (a_i[i] + distance_sum)

    return value

x_test_ga = np.array([2.5, 2.5])
f_test_ga = ga_objective_function(x_test_ga)

print("GA variant:", ga_variant)
print("Dimension n:", ga_n)
print("alpha =", ga_alpha)
print("beta =", ga_beta)
print("Test point:", x_test_ga)
print("Function value at test point:", f_test_ga)
```

```
... GA variant: 2
Dimension n: 2
alpha = 0
beta = 5
Test point: [2.5 2.5]
Function value at test point: 1.4379986467497168
```

Рисунок 15 — Задание параметров варианта для генетического алгоритма

### 3.10. Определение функции приспособленности и проверка допустимости решений

Так как генетический алгоритм выполняет отбор особей на основе функции приспособленности, была определена fitness-функция. В данной работе задача состоит в минимизации целевой функции, поэтому функция приспособленности была задана в виде обратного преобразования: чем меньше значение целевой функции, тем больше значение fitness.

Также были реализованы функции проверки допустимости решений и вычисления нарушения границ области поиска. Для проверки были рассмотрены несколько тестовых точек. Точки, координаты которых находятся в пределах  $[0, 5]$ , являются допустимыми. Точки, выходящие за границы допустимой области, имеют ненулевое нарушение и не допускаются как корректные решения задачи.

```
▶ # Определение функции приспособленности и проверка допустимости решений

def ga_is_feasible(x):
    x = np.array(x, dtype=float)
    return np.all(x >= ga_lower_bound) and np.all(x <= ga_upper_bound)

def ga_boundary_violation(x):
    x = np.array(x, dtype=float)
    lower_violation = np.maximum(ga_lower_bound - x, 0)
    upper_violation = np.maximum(x - ga_upper_bound, 0)
    return np.sum(lower_violation ** 2 + upper_violation ** 2)

def fitness_function(x):
    return 1.0 / (1.0 + ga_objective_function(x))

ga_test_points = [
    np.array([2.5, 2.5]),
    np.array([0.0, 0.0]),
    np.array([5.0, 5.0]),
    np.array([-1.0, 2.0]),
    np.array([6.0, 2.0])
]

for point in ga_test_points:
    print("Point:", point)
    print("f(x):", ga_objective_function(point))
    print("Fitness:", fitness_function(point))
    print("Boundary violation:", ga_boundary_violation(point))
    print("Feasible:", ga_is_feasible(point))
    print("-" * 50)
```

Рисунок 16 — Код определения функции приспособленности

```

... Point: [2.5 2.5]
f(x): 1.4379986467497168
Fitness: 0.41017250002709266
Boundary violation: 0.0
Feasible: True
-----
Point: [0. 0.]
f(x): 0.7086620515499825
Fitness: 0.5852532389847763
Boundary violation: 0.0
Feasible: True
-----
Point: [5. 5.]
f(x): 0.7683320620391557
Fitness: 0.5655046478356831
Boundary violation: 0.0
Feasible: True
-----
Point: [-1. 2.]
f(x): 0.5473054889153961
Fitness: 0.6462847880808353
Boundary violation: 1.0
Feasible: False
-----
Point: [6. 2.]
f(x): 0.6182641432641434
Fitness: 0.6179460900511182
Boundary violation: 1.0
Feasible: False
-----

```

Рисунок 17 — Результат проверки функции приспособленности

### 3.11. Формирование начальной популяции генетического алгоритма

На следующем этапе была сформирована начальная популяция генетического алгоритма. Размер популяции был выбран равным 40 особям, что соответствует рекомендуемому диапазону размера популяции для генетических алгоритмов.

Каждая особь представляет собой вектор из двух координат  $x_1$  и  $x_2$ . Координаты особей были сгенерированы случайным образом с равномерным распределением в допустимой области  $[0, 5]$ . Для каждой особи были вычислены значение целевой функции  $f(x)$  и значение функции приспособленности.

После формирования популяции была выполнена проверка допустимости всех особей. Результат проверки показал, что все элементы начальной популяции принадлежат заданной области поиска.

```
# Формирование начальной популяции для генетического алгоритма

population_size = 40

initial_population = np.random.uniform(
    low=ga_lower_bound,
    high=ga_upper_bound,
    size=(population_size, ga_n)
)

initial_objective_values = np.array([
    ga_objective_function(individual) for individual in initial_population
])

initial_fitness_values = np.array([
    fitness_function(individual) for individual in initial_population
])

initial_population_df = pd.DataFrame({
    "x1": initial_population[:, 0],
    "x2": initial_population[:, 1],
    "f(x)": initial_objective_values,
    "fitness": initial_fitness_values
})

print("Population size:", population_size)
print("All individuals are feasible:", np.all([ga_is_feasible(individual) for individual in initial_population]))
print("Best initial objective value:", initial_objective_values.min())
print("Best initial individual:", initial_population[initial_objective_values.argmin()])

display(initial_population_df.head(10))
```

Рисунок 18 — Код формирования начальной популяции

```

... Population size: 40
All individuals are feasible: True
Best initial objective value: 0.7284890344501075
Best initial individual: [0.45144885 4.17651248]

```

|   | x1       | x2       | f(x)     | fitness  |
|---|----------|----------|----------|----------|
| 0 | 0.597971 | 3.566224 | 0.834830 | 0.545010 |
| 1 | 3.803925 | 2.806386 | 1.188337 | 0.456968 |
| 2 | 3.854836 | 2.468978 | 1.152557 | 0.464564 |
| 3 | 2.613664 | 2.137705 | 1.432151 | 0.411159 |
| 4 | 0.127096 | 0.539457 | 0.952882 | 0.512064 |
| 5 | 0.157146 | 3.182052 | 0.769269 | 0.565205 |
| 6 | 1.571780 | 2.542853 | 1.351270 | 0.425302 |
| 7 | 4.537832 | 1.246461 | 0.834666 | 0.545058 |
| 8 | 2.051915 | 3.777756 | 1.110441 | 0.473835 |
| 9 | 1.143991 | 0.384900 | 1.291546 | 0.436387 |

Рисунок 19 — Начальная популяция генетического алгоритма

### 3.12. Определение операторов генетического алгоритма

После формирования начальной популяции были реализованы основные операторы генетического алгоритма: отбор, кроссинговер и мутация.

Оператор отбора выбирает особи для дальнейшего скрещивания с учетом значений функции приспособленности. Особи с большим значением *fitness* имеют большую вероятность быть выбранными. Оператор кроссинговера формирует двух потомков как линейные комбинации двух родительских особей. Такой способ скрещивания сохраняет допустимость потомков при условии, что родители находятся внутри выпуклой области допустимых значений.

Оператор мутации случайным образом изменяет отдельные координаты особи. Данная операция необходима для поддержания разнообразия популяции и уменьшения вероятности преждевременной сходимости алгоритма к локальному экстремуму.



### ▶ # Определение операторов отбора, кроссинговера и мутации

```
def selection(population, fitness_values):
    probabilities = fitness_values / np.sum(fitness_values)
    selected_indices = np.random.choice(
        len(population),
        size=len(population),
        replace=True,
        p=probabilities
    )
    return population[selected_indices]

def crossover(parent1, parent2, crossover_probability):
    if np.random.rand() < crossover_probability:
        c = np.random.rand()
        child1 = c * parent1 + (1 - c) * parent2
        child2 = (1 - c) * parent1 + c * parent2
        return child1, child2
    else:
        return parent1.copy(), parent2.copy()

def mutation(individual, mutation_probability):
    mutant = individual.copy()

    for j in range(len(mutant)):
        if np.random.rand() < mutation_probability:
            mutant[j] = np.random.uniform(ga_lower_bound, ga_upper_bound)

    return mutant

print("Selection, crossover and mutation operators are defined successfully")
```

```
... Selection, crossover and mutation operators are defined successfully
```

Рисунок 20 — Определение операторов генетического алгоритма

### 3.13. Реализация генетического алгоритма

На данном этапе была реализована базовая версия генетического алгоритма. Перед запуском были заданы основные параметры алгоритма: количество поколений равно 100, вероятность кроссинговера  $P_c = 0.4$ , вероятность мутации  $P_m = 0.1$ , размер популяции равен 40 особям.

На каждой итерации алгоритма вычислялись значения целевой функции и функции приспособленности для всех особей текущей популяции. После этого выполнялся отбор особей, кроссинговер родительских пар и мутация потомков. Для сохранения допустимости решений координаты новых особей ограничивались областью  $[0, 5]$ .

В результате работы базового генетического алгоритма было найдено допустимое приближенное решение задачи. Найденная точка принадлежит области допустимых значений, а величина нарушения границ равна нулю.

▶ # Реализация генетического алгоритма для решения задачи многоэкстремальной оптимизации

```
number_of_generations = 100
crossover_probability = 0.4
mutation_probability = 0.1

population = initial_population.copy()

best_history = []
mean_history = []

start_time = time.time()

for generation in range(number_of_generations):
    objective_values = np.array([
        ga_objective_function(individual) for individual in population
    ])

    fitness_values = np.array([
        fitness_function(individual) for individual in population
    ])

    best_index = np.argmin(objective_values)
    best_history.append(objective_values[best_index])
    mean_history.append(objective_values.mean())

    selected_population = selection(population, fitness_values)

    new_population = []
```



```

new_population = []

for i in range(0, population_size, 2):
    parent1 = selected_population[i]
    parent2 = selected_population[(i + 1) % population_size]

    child1, child2 = crossover(parent1, parent2, crossover_probability)

    child1 = mutation(child1, mutation_probability)
    child2 = mutation(child2, mutation_probability)

    child1 = np.clip(child1, ga_lower_bound, ga_upper_bound)
    child2 = np.clip(child2, ga_lower_bound, ga_upper_bound)

    new_population.append(child1)
    new_population.append(child2)

population = np.array(new_population[:population_size])

ga_execution_time = time.time() - start_time

final_objective_values = np.array([
    ga_objective_function(individual) for individual in population
])

best_final_index = np.argmin(final_objective_values)
best_ga_solution = population[best_final_index]
best_ga_value = final_objective_values[best_final_index]

print("Genetic algorithm completed")
print("Number of generations:", number_of_generations)
print("Population size:", population_size)
print("Crossover probability:", crossover_probability)
print("Mutation probability:", mutation_probability)
print("Best solution:", best_ga_solution)
print("Best function value:", best_ga_value)
print("Execution time:", ga_execution_time)
print("Feasible:", ga_is_feasible(best_ga_solution))
print("Boundary violation:", ga_boundary_violation(best_ga_solution))

```

Рисунок 21 — Код реализации генетического алгоритма

```

... Genetic algorithm completed
Number of generations: 100
Population size: 40
Crossover probability: 0.4
Mutation probability: 0.1
Best solution: [2.43884975 0.01823351]
Best function value: 0.8601526604558493
Execution time: 0.5495190620422363
Feasible: True
Boundary violation: 0.0

```

Рисунок 22 — Результат работы генетического алгоритма

### 3.14. Анализ сходимости генетического алгоритма

Для анализа поведения базового генетического алгоритма был построен график изменения лучшего и среднего значения целевой функции по поколениям. По оси абсцисс отложен номер поколения, а по оси ординат — значение функции  $f(x)$ .

Из графика видно, что среднее значение целевой функции изменяется в процессе эволюции популяции, а лучшее значение может улучшаться неравномерно. Это связано с тем, что в базовой версии алгоритма лучшая найденная особь не сохраняется гарантированно при переходе к следующему поколению. Поэтому после операций отбора, кроссинговера и мутации хорошее решение может быть потеряно.

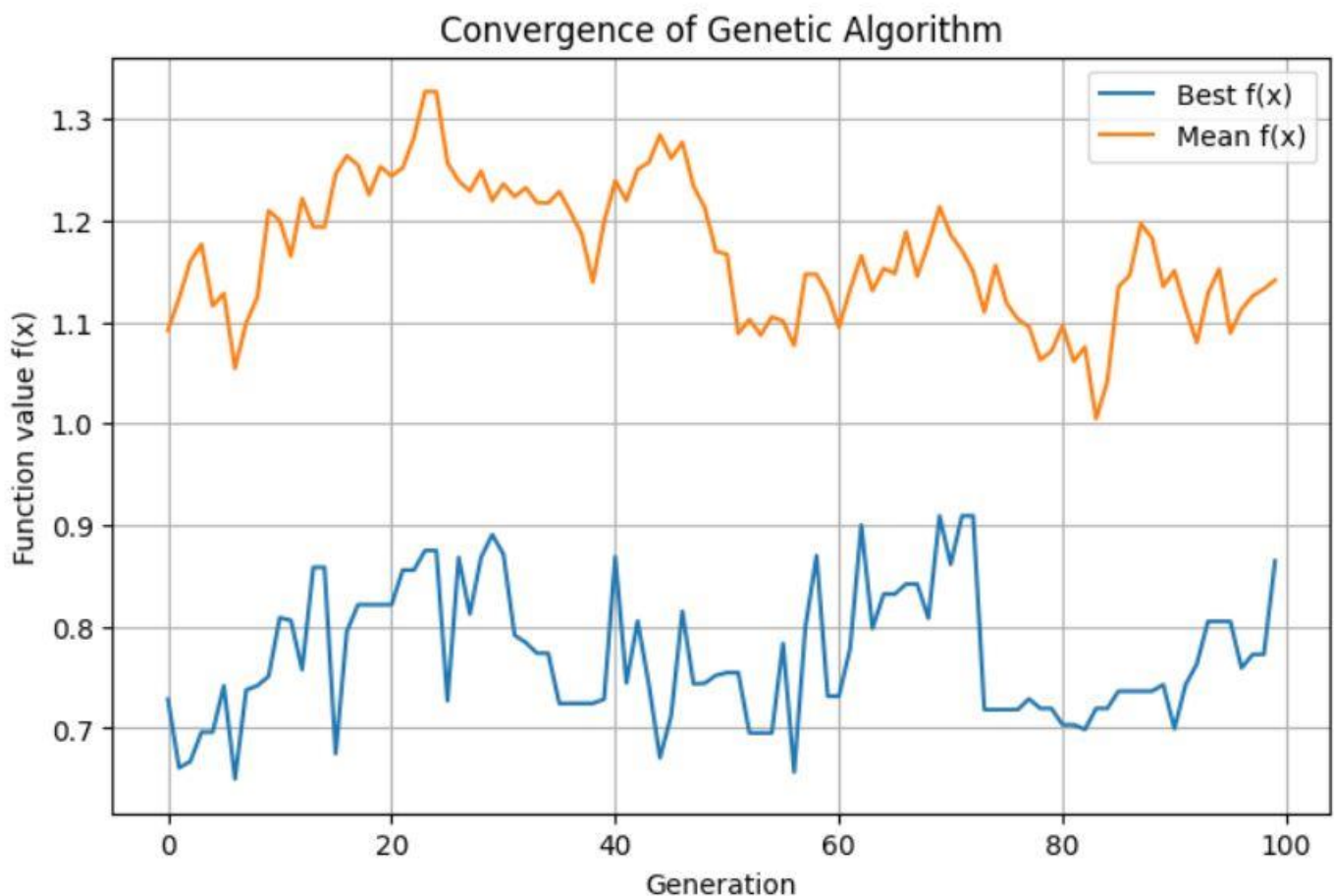


Рисунок 23 — График сходимости генетического алгоритма

### 3.15. Реализация генетического алгоритма с сохранением лучшей особи

Для повышения устойчивости генетического алгоритма была реализована модификация с сохранением лучшей особи. Данный подход называется elitism. Его основная идея состоит в том, что лучшее найденное решение обязательно переносится в следующее поколение и не может быть потеряно в результате кроссинговера или мутации.

В реализации на каждой итерации определялась лучшая особь текущей популяции. Если ее значение целевой функции было меньше глобально лучшего найденного значения, то это решение сохранялось как новое лучшее. При формировании следующего поколения сохраненная лучшая особь добавлялась первой, а остальные элементы популяции формировались с помощью стандартных операторов отбора, кроссинговера и мутации.

В результате генетический алгоритм с сохранением лучшей особи нашел лучшее значение целевой функции по сравнению с базовой версией алгоритма. Найденное решение принадлежит допустимой области, а величина нарушения границ равна нулю.

```
▶ # Реализация генетического алгоритма с сохранением лучшей особи

elite_population = initial_population.copy()

elite_best_history = []
elite_mean_history = []

global_best_solution = None
global_best_value = np.inf

start_time = time.time()

for generation in range(number_of_generations):
    objective_values = np.array([
        ga_objective_function(individual) for individual in elite_population
    ])

    fitness_values = np.array([
        fitness_function(individual) for individual in elite_population
    ])

    best_index = np.argmin(objective_values)
    current_best_solution = elite_population[best_index].copy()
    current_best_value = objective_values[best_index]

    if current_best_value < global_best_value:
        global_best_value = current_best_value
        global_best_solution = current_best_solution.copy()

    elite_best_history.append(global_best_value)
```

```

elite_best_history.append(global_best_value)
elite_mean_history.append(objective_values.mean())

selected_population = selection(elite_population, fitness_values)

new_population = [global_best_solution.copy()]

while len(new_population) < population_size:
    parent_indices = np.random.choice(len(selected_population), size=2, replace=False)
    parent1 = selected_population[parent_indices[0]]
    parent2 = selected_population[parent_indices[1]]

    child1, child2 = crossover(parent1, parent2, crossover_probability)

    child1 = mutation(child1, mutation_probability)
    child2 = mutation(child2, mutation_probability)

    child1 = np.clip(child1, ga_lower_bound, ga_upper_bound)
    child2 = np.clip(child2, ga_lower_bound, ga_upper_bound)

    new_population.append(child1)
    if len(new_population) < population_size:
        new_population.append(child2)

elite_population = np.array(new_population)

elite_ga_execution_time = time.time() - start_time

print("Genetic algorithm with elitism completed")
print("Number of generations:", number_of_generations)
print("Population size:", population_size)
print("Crossover probability:", crossover_probability)
print("Mutation probability:", mutation_probability)
print("Best solution:", global_best_solution)
print("Best function value:", global_best_value)
print("Execution time:", elite_ga_execution_time)
print("Feasible:", ga_is_feasible(global_best_solution))
print("Boundary violation:", ga_boundary_violation(global_best_solution))

```

Рисунок 24 — Код генетического алгоритма с сохранением лучшей особи



```
... Genetic algorithm with elitism completed
Number of generations: 100
Population size: 40
Crossover probability: 0.4
Mutation probability: 0.1
Best solution: [0.03476065 4.90832825]
Best function value: 0.5788257244560986
Execution time: 1.3790788650512695
Feasible: True
Boundary violation: 0.0
```

Рисунок 25 — Результат генетического алгоритма с сохранением лучшей особи

### 3.16. Анализ сходимости генетического алгоритма с сохранением лучшей особи

После реализации генетического алгоритма с сохранением лучшей особи был построен график сходимости. На графике отображаются лучшее найденное значение целевой функции и среднее значение функции в популяции по поколениям.

Из графика видно, что лучшее значение целевой функции не увеличивается при переходе от одного поколения к другому. Это объясняется использованием механизма elitism, при котором лучшая найденная особь сохраняется и переносится в следующее поколение. Благодаря этому процесс поиска становится более устойчивым, а найденное решение не теряется в ходе дальнейшей эволюции популяции.

Среднее значение целевой функции продолжает изменяться, так как остальные особи популяции подвергаются отбору, кроссинговеру и мутации. Однако сохранение лучшей особи позволяет получить более качественный итоговый результат по сравнению с базовой версией генетического алгоритма.

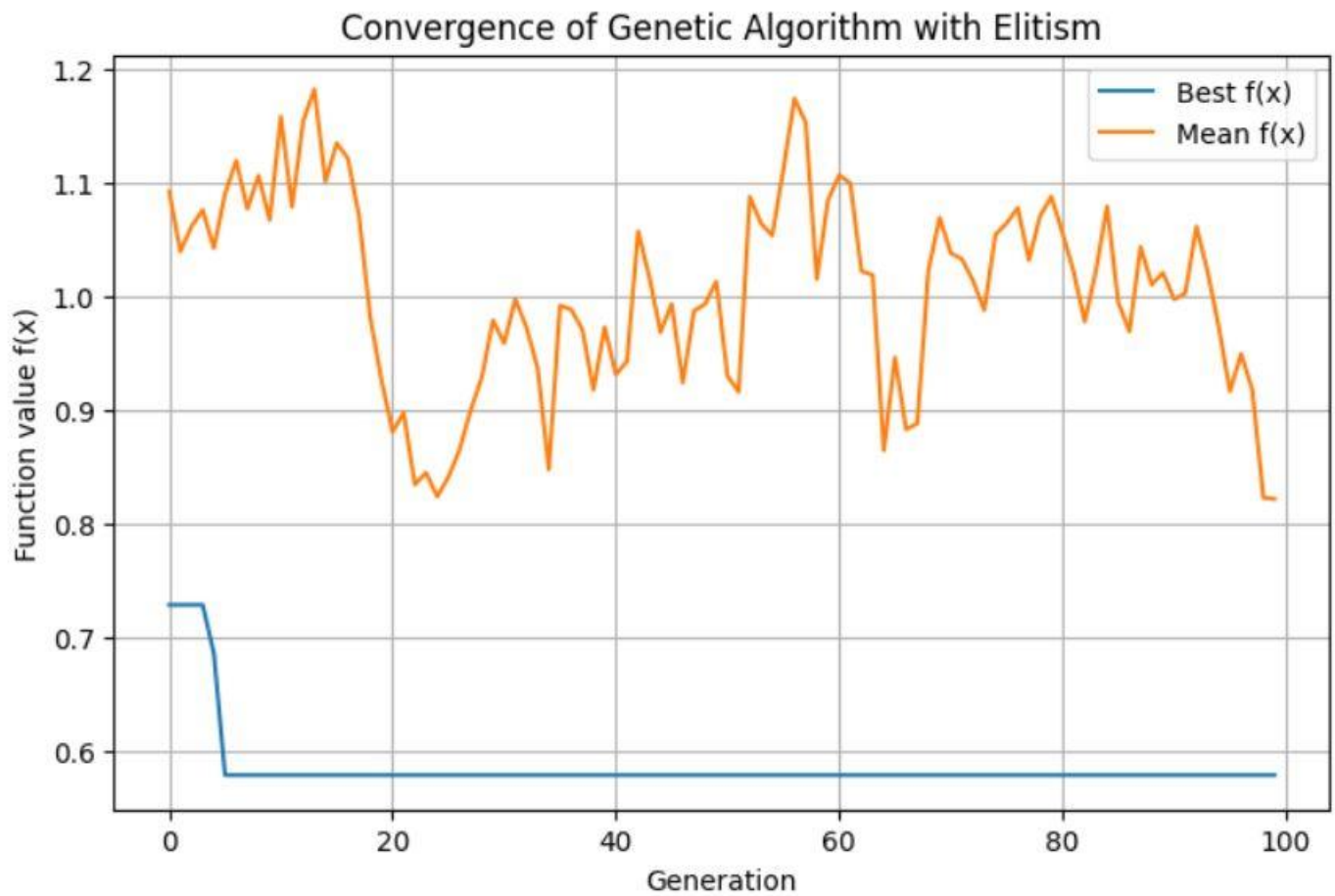


Рисунок 26 — График сходимости генетического алгоритма с сохранением лучшей особи



### 3.17. Итоговое сравнение результатов

На заключительном этапе была сформирована итоговая сравнительная таблица результатов. В таблицу были включены три реализованных подхода: метод случайного мултистарта, базовый генетический алгоритм и генетический алгоритм с сохранением лучшей особи.

Для каждого метода были указаны найденные координаты решения, значение целевой функции, количество запусков или поколений, время выполнения, признак допустимости решения и величина нарушения границ области поиска. Это позволяет сравнить методы не только по качеству найденного решения, но и по корректности выполнения ограничений.

По результатам вычислений метод случайного мултистарта для функции Шекеля нашел точку  $x = (3, 3, 3)$  со значением  $f(x) = -1.5$ . Для задачи генетического алгоритма обе версии алгоритма нашли допустимые решения, однако модификация с сохранением лучшей особи показала лучшее значение целевой функции по сравнению с базовой версией. Все найденные решения удовлетворяют ограничениям соответствующих областей поиска.

|   | Method                         | Best x1  | Best x2  | Best x3 | Best f(x) | Iterations / generations | Execution time | Feasible | Boundary violation |
|---|--------------------------------|----------|----------|---------|-----------|--------------------------|----------------|----------|--------------------|
| 0 | Random Multistart              | 3.000000 | 3.000000 | 3.0     | -1.500000 | 30                       | 0.173834       | True     | 0.0                |
| 1 | Genetic Algorithm              | 2.438850 | 0.018234 | -       | 0.860153  | 100                      | 0.549519       | True     | 0.0                |
| 2 | Genetic Algorithm with Elitism | 0.034761 | 4.908328 | -       | 0.578826  | 100                      | 1.379079       | True     | 0.0                |

Рисунок 27 — Итоговая сравнительная таблица результатов

#### 4) Заключение

В ходе лабораторной работы были изучены и реализованы методы решения задач многоэкстремальной оптимизации. В первой части работы был реализован метод случайного мультистарта для функции Шекеля. Для выбранного варианта были заданы параметры функции и область допустимых значений, после чего была выполнена локальная оптимизация из нескольких случайных начальных точек.

По результатам вычислений метод случайного мультистарта нашел глобальный минимум функции Шекеля в точке  $x = (3, 3, 3)$ , где значение функции равно  $f(x) = -1.5$ . Все найденные решения удовлетворяют ограничениям области поиска. Дополнительно была выполнена кластеризация найденных точек методом ближайшего соседа, которая показала, что все запуски локальной оптимизации сошлись к одному кластеру.

Во второй части работы был реализован генетический алгоритм для многоэкстремальной функции варианта 2. Были выполнены этапы формирования начальной популяции, вычисления функции приспособленности, отбора, кроссинговера и мутации. Также была реализована модификация генетического алгоритма с сохранением лучшей особи. Данная модификация позволила получить более устойчивую сходимость и лучшее значение целевой функции по сравнению с базовой версией алгоритма.

Таким образом, все поставленные задачи лабораторной работы были выполнены. Реализованные методы позволили найти допустимые приближенные решения задач многоэкстремальной оптимизации и сравнить полученные результаты по значению целевой функции, времени выполнения и выполнению ограничений.